

π Tantum

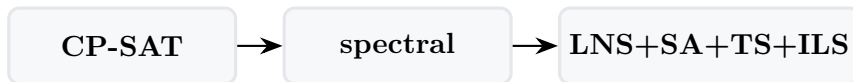
alias *Tempus Tantum*

sistema di generazione orario scolastico

“Omnia, Lucili, aliena sunt, tempus tantum nostrum est.”
Seneca, *Epistulae morales ad Lucilium*, I, 1

Manuale completo

Parte I (manuale generale) + Parte II (appendici tecniche)



Giovanni Borghi

3 maggio 2026

Repository: <https://github.com/gborghi/timetable>
Documentazione sorgente: docs/

Indice

I	Manuale generale	5
1	Introduzione	6
1.1	Avvio rapido	7
1.2	Stato attuale del progetto	7
2	Installazione	8
2.1	Windows	8
2.2	Linux (Ubuntu, Debian, Fedora, Arch)	9
2.3	macOS (Intel + Apple Silicon)	10
2.4	Verifica installazione	10
2.5	Aggiornamento	11
2.6	Disinstallazione	11
3	Vincoli	12
3.1	I 5 stati / 4 kind	12
3.2	Matrici di disponibilità 5-stati	12
3.3	Vincoli logici (DNF)	12
3.3.1	Sintassi	12
3.3.2	Predicate atoms	13
3.3.3	Esempi pratici	13
3.3.4	Mapping CP-SAT	13
3.4	Vincoli HARD a toggle (per classe)	14
3.5	Conflict detection	14
3.6	Tag aule – etichette libere	14
3.7	Preferenze di giorno libero del docente	14
3.8	Preferenze di giorno libero della classe	15
4	Workflow di ottimizzazione	16
4.1	Schema delle fasi	16
4.2	Phase A: assegnazione (cattedre)	16
4.3	Phase B: scheduling	16
4.4	Metaeuristiche	16
4.5	Phase 4: assegnazione aule	17
4.6	Drag-and-drop con preview live	17
4.6.1	Room-follows-lesson	17
4.7	Slot picker matriciale (Monitor)	18
4.8	Punteggio di graduatoria e criterio Anzianita	18

5	Guida UI	19
5.1	Funzionalità trasversali	19
5.1.1	Query DSL e sort multi-livello	19
5.1.2	Multi-select	19
5.1.3	Excel/CSV import	19
5.1.4	Esportazione liste (xlsx / csv)	19
5.1.5	Viste salvate	20
5.1.6	Tag (aule e studenti)	20
5.1.7	Import bulk dedicato (/import)	20
5.1.8	Import / Export DB (Dashboard)	20
5.2	Tab per tab	21
6	Launcher	22
6.1	Windows: start.bat	22
6.2	Linux / macOS: start.sh + stop.sh	22
II	Appendici tecniche	23
7	Architettura	24
7.1	Stack	24
7.2	Diagramma a blocchi	24
7.3	Struttura cartelle	24
7.4	Lifecycle del backend	25
8	Modello dati	26
8.1	Famiglie di tabelle	26
8.2	Diagramma relazioni	26
8.3	Migrazioni idempotenti	26
8.4	Pickle dell'engine	27
9	DSL generico per i vincoli	28
9.1	Filosofia	28
9.2	Grammatica BNF	28
9.3	Sorgenti iterabili	29
9.4	Galleria di esempi	29
9.4.1	Vincoli docente (10 esempi)	29
9.4.2	Vincoli classe (5 esempi)	30
9.4.3	Vincoli aula (5 esempi)	31
9.4.4	Vincoli materia (3 esempi)	32
9.4.5	Vincoli gruppo / studenti (3 esempi)	32
9.4.6	Vincoli relazionali / coupling (4 esempi)	32
9.5	Errori comuni	33
10	Tecniche di ottimizzazione avanzate	34
10.1	Adaptive Large Neighborhood Search (ALNS)	34
10.2	Variable Neighborhood Search (VNS)	34
10.3	Hall's theorem pre-check	34
10.4	Column Generation (Dantzig-Wolfe)	34
10.5	Lagrangian Relaxation	34

11 Diagnostica statistica	35
11.1 Sensitivity Monte Carlo	35
11.2 Analisi bipartito	35
11.3 Correlazioni e regressioni	35
11.4 Distribuzioni	35
11.5 Run telemetry	35
12 API REST	36
12.1 Pattern CRUD	36
12.2 Esempi curl	36
12.3 Riferimento gruppi di endpoint	37
13 Estendere il sistema	38
13.1 Aggiungere una nuova tabella + CRUD	38
13.2 Migrazione idempotente	38
13.3 Aggiungere un nuovo tipo di vincolo	38
13.4 Aggiungere un nuovo step di ottimizzazione	38
13.5 Testing	39
A Repository GitHub	40
B Glossario discorsivo dei metodi	41
B.1 CP-SAT (Constraint Programming over SAT)	41
B.2 Decomposizione spettrale	41
B.3 LNS (Large Neighborhood Search)	42
B.4 ALNS (Adaptive LNS)	42
B.5 SA (Simulated Annealing)	43
B.6 TS (Tabu Search)	43
B.7 ILS (Iterated Local Search)	43
B.8 VNS (Variable Neighborhood Search)	43
B.9 Hall's theorem pre-check	44
B.10 Lagrangian Relaxation con sub-gradient	44
B.11 Column Generation (Dantzig-Wolfe)	45
B.12 Monte Carlo Sensitivity	45
B.13 Modularità, betweenness, densità (analisi del bipartito)	45
B.14 Distribuzioni e istogrammi	46
B.15 Correlazioni e regressioni	46
B.16 DSL generico	46
B.17 Stati 5-colori delle matrici	47
B.18 Tag aule e tag studenti	47
B.19 Run telemetry e visualizzazione live	48
B.20 Lockati: lezioni che non si toccano	48
C Reference	49

Elenco delle figure

4.1	Pipeline a 4 fasi.	16
7.1	Architettura a blocchi della webapp.	24
8.1	Relazioni rilevanti del modello dati (semplificato).	27

Parte I

Manuale generale

1. Introduzione

Come usare questo manuale

La documentazione di π Tantum è suddivisa in due parti, per adattarsi a pubblici diversi:

- Il **Manuale generale** (Parte I, file `manual_generale.pdf`) è pensato per chi usa π Tantum come strumento di lavoro: dirigenti scolastici, coordinatori d'orario, docenti che devono modificare un vincolo o capire perché un orario è venuto in un certo modo. Tono discorsivo, casi d'uso concreti, niente jargon.
- Le **Appendici tecniche** (Parte II, file `appendici_tecniche.pdf`) sono per sviluppatori, amministratori IT e curiosi: architettura interna, modello dati, API REST, grammatica formale del DSL, dettaglio delle strategie di ottimizzazione e della diagnostica statistica.
- Il **Manuale completo** (file `manual_completo.pdf`) raccoglie entrambe le parti in un unico volume, utile per chi vuole stamparlo o averlo tutto a portata di mano.

Tutti e tre i PDF sono ricostruibili dai sorgenti `docs/*.tex` con lo script `docs/build_manual.sh` (o `.bat` su Windows).

Cos'è π Tantum

π Tantum (alias *Tempus Tantum*, codename interno: `timetable`) è un sistema di generazione e gestione dell'orario scolastico per un Liceo italiano. Il nome richiama il celebre incipit dell'epistola I di Seneca a Lucilio: “Omnia, Lucili, aliena sunt, tempus tantum nostrum est” (Seneca, *Epistulae morales ad Lucilium*, I, 1) – “tutto ci appartiene di altri, soltanto il tempo è nostro”. Il gioco grafico fra la lettera greca π e le due “T” di *Tempus Tantum* riassume l'idea: organizzare il tempo della scuola, dato che del tempo non si dispone se non gestendolo.

È composto di tre layer:

1. **Solver** (`experiments/`): pipeline CP-SAT (Google OR-Tools) con decomposizione spettrale per istanze grandi e cascata metaeuristica (LNS, SA, TS, ILS).
2. **Web UI** (`webui/`): backend FastAPI su SQLite + frontend SvelteKit. È il punto di accesso quotidiano: gestisce CRUD, ottimizzazione, visualizzazione, drag-and-drop, assenze e supplenze.
3. **Legacy** (`schedule/`): script monolitici da cui il solver è nato; mantenuti per riferimento storico.

Il manuale copre tutti e tre i layer ma si concentra sulla web UI, che ha assorbito quasi ogni funzionalità operativa.

1.1 Avvio rapido

Windows. Doppio click su `webui/start.bat`. Apre due finestre cmd: backend *uvicorn* su porta 8000, frontend *vite* su porta 5173.

Linux / macOS.

```
cd timetable/webui
./start.sh # background; URLs sulla console
./stop.sh # ferma backend + frontend
```

Nel browser, aprire `http://127.0.0.1:5173` e dalla **Dashboard** importare un profilo già calcolato (`small` per cominciare).

1.2 Stato attuale del progetto

Le funzionalità principali, al momento di questa redazione:

- 16 sezioni (tab) della UI, ~118 endpoint REST.
- ~30 tabelle SQLAlchemy con migrazioni idempotenti.
- 5 stati di vincoli (HARD/SOFT/PREFERITO/ENFORCED/ALLOWED).
- Vincoli logici DNF con DSL esteso (predicate atoms `aula:`, `materia:`, `classe:`, `gruppo:`, `docente:`).
- Conflict detection automatico nel tab *Vincoli*.
- Drag-and-drop con preview live (rosso/giallo/verde) e *room-follows-lesson*.
- Gestione assenze settimanali con assegnazione supplenti drag-drop.
- Tab *Monitor* con espansione lezione-per-lezione e slot picker matriciale.

2. Installazione

Guida cross-platform per installare ed eseguire piTantum su Windows, Linux e macOS. La versione *di riferimento* di questa guida è `docs/installation.md` nel repo (Markdown, sempre aggiornato); qui se ne riproduce il contenuto in forma compatta per il manuale PDF. In caso di divergenza, fa fede il file Markdown.

Architettura del lancio. Il progetto è composto da due processi:

- **Backend:** `uvicorn` che serve l'app FastAPI in `webui/backend` sulla porta 8000.
- **Frontend:** server di sviluppo `vite` che serve l'app SvelteKit in `webui/frontend` sulla porta 5173.

Uno script unico (`start.bat` su Windows, `start.sh` su Linux/macOS) li avvia entrambi. Al primo lancio crea la `venv` Python, installa i requirements, fa `npm install`; ai lanci successivi parte in pochi secondi.

2.1 Windows

Prerequisiti.

- Python ≥ 3.11 – installer ufficiale: <https://www.python.org/downloads/windows/>
- Node.js ≥ 20 LTS – installer ufficiale: <https://nodejs.org/en/download> (scegliere “LTS”).
- Git – installer ufficiale: <https://git-scm.com/download/win>.

Note importanti.

- Sull'installer Python, **spuntare** “Add python.exe to PATH”. Senza, lo script non trova il comando `python`.
- Riavviare il terminale dopo le installazioni: il PATH non si aggiorna nelle finestre già aperte.

Clone + lancio.

```
git clone https://github.com/gborghi/timetable.git
cd timetable
webui\start.bat
```

Lo script apre due finestre cmd (“piTantum – backend” e “piTantum – frontend”). Aprire il browser su `http://localhost:5173`.

Stop. Chiudere le due finestre cmd, oppure `Ctrl+C` in ciascuna.

Troubleshooting Windows.

- “python non riconosciuto”: l’installer Python non è stato eseguito con “Add to PATH”. Reinstallare oppure aggiungere C:\Users\tu\AppData\Local\Programs\Python\Python311 al PATH.
- Porta 8000 occupata: `netstat -ano | findstr :8000 + taskkill /PID <pid> /F`.
- Antivirus blocca `start.bat`: aggiungere alle eccezioni di Windows Defender.

2.2 Linux (Ubuntu, Debian, Fedora, Arch)

Prerequisiti. Python ≥ 3.11 , Node ≥ 20 LTS, Git, build-essential.

Ubuntu / Debian.

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip nodejs npm \
git build-essential
```

I repo apt di Ubuntu 22.04 / Debian 12 hanno Node *12-18*, troppo vecchio per SvelteKit 2 (richiede 20+). In quel caso usare **nvm**:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install --lts && nvm use --lts
```

Fedora.

```
sudo dnf install -y python3 python3-virtualenv nodejs npm git \
gcc make
```

Arch / Manjaro.

```
sudo pacman -S python python-pip nodejs npm git base-devel
```

Clone + lancio.

```
git clone https://github.com/gborghi/timetable.git
cd timetable
chmod +x webui/start.sh webui/stop.sh
./webui/start.sh # background; log in webui/logs/
./webui/start.sh --foreground # log live in console (Ctrl+C ferma)
```

Stop.

```
./webui/stop.sh
```

Troubleshooting Linux.

- Porta in uso: `sudo lsof -i :8000 + kill <pid>` (eventualmente `kill -9 <pid>`).
- python3 mancante su distro minimal: usare l’eseguibile chiamato `python` (3.x).
- WSL2: `start.sh` funziona normalmente. Aprire il browser dal lato Windows su `http://localhost:5173`; WSL2 inoltra le porte automaticamente.

2.3 macOS (Intel + Apple Silicon)

Prerequisiti.

- Xcode Command Line Tools: `xcode-select -install`.
- Homebrew: vedere <https://brew.sh/>.
- Python ≥ 3.11 , Node ≥ 20 LTS, Git: tutti via Homebrew.

Installazione.

```
# Homebrew (se non presente)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Toolchain
brew install python@3.12 node git

# Verifica
python3 --version # >= 3.11
node --version # >= v20
```

Note Apple Silicon.

- ortools dalla v9.7 spedisce wheel ARM64 nativi: Rosetta non serve.
- Se Python è installato da python.org invece che Homebrew, scegliere l'installer “macOS 64-bit universal2”.

Clone + lancio + stop. Identico a Linux.

Troubleshooting macOS.

- Gatekeeper blocca `start.sh`: `xattr -d com.apple.quarantine webui/start.sh webui/stop.sh`.
- ortools senza wheel ARM (versioni vecchie): `pip install -upgrade ortools` dentro la venv del backend.
- brew non in PATH: `eval "$(/opt/homebrew/bin/brew shellenv)"` in `~/.zprofile`.

2.4 Verifica installazione

Dopo il primo lancio andato a buon fine:

```
python --version # >= 3.11 (Linux/macOS: spesso python3)
node --version # >= v20
npm --version # >= 10

# Backend health
curl http://127.0.0.1:8000/api/health
# atteso: {"status":"ok","name":"pitantum","version":"0.1.0"}

# Async layer (Section 2.5 P2)
curl http://127.0.0.1:8000/api/health/async
# atteso: {"status":"ok","async":true,"dialect":"sqlite"}

# Frontend
curl -I http://127.0.0.1:5173
# atteso: HTTP/1.1 200 OK
```

2.5 Aggiornamento

Quando si fa `git pull` per portare a casa nuovi commit:

```
cd /path/to/timetable
git pull origin main

# se sono cambiate dipendenze Python:
webui/backend/.venv/bin/pip install -r webui/backend/requirements.txt
# (Windows: webui\backend\.venv\Scripts\pip install ...)

# se sono cambiate dipendenze frontend:
cd webui/frontend && npm install && cd ../../

# se ci sono migration DB:
cd webui/backend && ../.venv/bin/alembic upgrade head && cd ../../
# (Windows: webui\backend\.venv\Scripts\alembic upgrade head)
```

In pratica `start.sh/start.bat` rilanciato dopo un `git pull` gestisce automaticamente `pip install` e `npm install` quando servono. Le migration DB sono coperte da `_apply_lightweight_migrations()` allo startup come `safety-net` (vedi capitolo *Modello dati*); per la via canonica usare `alembic upgrade head`.

Dopo un aggiornamento backend, fare un **hard reload** del browser (`Ctrl+Shift+R`) per pulire chunk JavaScript cached.

2.6 Disinstallazione

```
cd /path/to/timetable
./webui/stop.sh # Linux/macOS
# (Windows: chiudere le finestre cmd di start.bat)

rm -rf webui/backend/.venv # ~150 MB
rm -rf webui/frontend/node_modules # ~400 MB
rm -rf webui/data/timetable.db # DATI UTENTE (salvarli prima!)
```

Per rimuovere completamente, cancellare la cartella `timetable/`. Python, Node, Git restano installati nel sistema; rimuoverli dal gestore di pacchetti solo se non servono ad altri progetti.

3. Vincoli

3.1 I 5 stati / 4 kind

Il sistema usa una tassonomia uniforme di *5 stati* sia per le matrici di disponibilità (cella per cella) sia per le preferenze di aula. I vincoli logici disgiuntivi (DNF) hanno *4 kind* (ALLOWED non si applica qui).

Stato	Colore	Semantica
ALLOWED	verde chiaro	default (solo grid materia↔aula e docente↔aula); nessuna contribuzione all'obiettivo
SOFT	giallo	penalità positiva quando lo slot è usato (+ penalty aggiunto all'obiettivo, che è minimizzato)
HARD	rosso	il solver deve evitare questo slot, oppure il vincolo logico DEVE essere soddisfatto
PREFERITO	blu	bonus negativo quando lo slot è usato / il vincolo logico è soddisfatto (- bonus riduce l'obiettivo)
ENFORCED	verde scuro	il solver DEVE far accadere questo slot / soddisfare il vincolo con presence required

3.2 Matrici di disponibilità 5-stati

Il ciclo cliccando una cella è:

`free → soft → hard → preferred → enforced → free`

Le celle gialle e blu mostrano un input numerico per editare la penalità (positiva sul giallo, negativa sul blu, sign-clamp automatico al salvataggio).

Sincronizzazione free_day. Quando il docente ha `free_day` settato, tutte e 6 le ore di quel giorno appaiono HARD nella matrice automaticamente. Se l'utente edita la matrice manualmente e tutte le ore di un giorno diventano HARD, il backend deduce il `free_day`; viceversa cambiare il `free_day` sostituisce le auto-cells.

Mapping CP-SAT. In `optimization.py::_availability_constraints` ogni tabella si traduce in: `<entity>_hard` (set di tuple, forbidden assignment), `<entity>_soft` (dict tuple→penalty signed), `<entity>_enforced` (set di tuple, presence required nel solver).

3.3 Vincoli logici (DNF)

3.3.1 Sintassi

```

expr := or_expr
or_expr := and_expr ('OR' and_expr)*
and_expr := not_expr ('AND' not_expr)*
not_expr := ('NOT' | 'mai') not_expr | atom
atom := slot | predicate | '(' expr ')'
slot := <giorno><ora>
predicate:= <kind> ':' <name> ('@' <giorno> <ora>)?
kind := aula | materia | classe | gruppo | docente | prof
giorno := lun|mar|mer|gio|ven|sab (anche lunedì|...|sabato,
        monday|...|saturday)
ora := 1..6 (ordinale, 1=08:00) | 8..13 (assoluta)

```

Sono accettati & (AND), | (OR), ! (NOT). mai è alias di NOT.

3.3.2 Predicate atoms

```

aula:LabFisica -- l'entita' usa l'aula a qualunque slot
aula:LabFisica@mar -- il martedì a qualunque ora
aula:LabFisica@mar4 -- martedì 4a ora (= 11:00)
materia:Religione@lun8 -- lezione di religione lunedì alle 8
classe:1A_Scientifico -- riferimento alla classe
gruppo:IRC -- riferimento al gruppo

```

I literal predicate vengono parsati e persistiti correttamente. Il solver li tratta otticamente per ora; l'integrazione strong con la fase classroom assignment è pianificata.

3.3.3 Esempi pratici

```

% Per docenti
lun8 AND lun9
(lun8 AND lun9) OR (mar8 AND mar9)
gio11 OR gio12 OR gio13
NOT (mer3 AND mer4)
mai aula:LabFisica@mar
mai materia:Religione@lun8

% Per classi
mer4 AND mer5 AND mer6
mai aula:Palestra@gio
aula:LabInformatica@mar OR aula:LabInformatica@gio

% Per aule
gio4 AND gio5 AND gio6
mai materia:Religione
gruppo:IRC OR gruppo:Alternativa

```

3.3.4 Mapping CP-SAT

Per ogni regola con DNF [clause1, clause2, ...]:

- HARD: il solver impone almeno una clausola fully active.
- SOFT: se nessuna clausola è fully active, paga +soft_penalty.
- PREFERITO: se almeno una clausola è fully active, ottiene +soft_penalty (negativo, quindi reduce dell'obiettivo).

- **ENFORCED**: come **HARD** ma con *presence required* – almeno uno degli slot della DNF deve coincidere con un’ora di lezione attiva per quell’entità.

3.4 Vincoli **HARD** a toggle (per classe)

In `school_classes` ci sono 7 boolean strutturali: `hard_entry_at_8`, `hard_exit_after_12`, `hard_no_holes`, `hard_dual_math`, `hard_dual_italian`, `hard_motorie_pairs`, `hard_max_6_per_day`. Il soft `soft_minimize_sixth_weight` regola il peso della 6a ora.

3.5 Conflict detection

Il tab *Vincoli* ha un bottone "Cerca conflitti" (GET `/api/monitor/conflicts`) che segnala:

- **matrix_hard_enforced** – stessa cella marcata sia **HARD** sia **ENFORCED**.
- **enforced_on_free_day** – cella **ENFORCED** del docente nel suo `free_day`.
- **logical_unsatisfiable** – vincolo logico **HARD/ENFORCED** la cui DNF non è soddisfacibile dato l’insieme degli **HARD** slot dell’owner.

3.6 Tag aule – etichette libere

Per esprimere “questa aula ha il proiettore”, “questa aula è adatta alla matematica perché ha la lavagna grande”, “questa aula è una sala riunioni”, il sistema offre i **tag delle aule**: etichette libere che il coordinatore attacca dalla scheda Aule. Ogni aula può avere quanti tag vuoi; ogni tag può essere riutilizzato su più aule.

I tag sono utilissimi nei vincoli: invece di scrivere “solo queste tre aule precise” (lista che andrebbe aggiornata ogni volta che le aule cambiano), scrivi “aule taggate matematica” e il vincolo segue automaticamente le etichettature correnti.

Esempio: vogliamo che la matematica si tenga solo in aule taggate `matematica`. Una volta etichettate le aule giuste dalla scheda Aule, il vincolo nel DSL generico è:

```
forall l in lessons where l.subject == Matematica:
    "matematica" in l.classroom.tags
```

Il sistema ha anche una serie di tag automatici prodotti dalla generazione mock (per esempio le aule home class ricevono il tag dell’indirizzo della classe assegnata). Per dettagli vedi il modulo `webui/backend/mock_classrooms.py`.

3.7 Preferenze di giorno libero del docente

I docenti hanno diritto contrattuale a un certo numero di giorni liberi a settimana (tipicamente uno per il CCNL italiano). π Tantum gestisce questo aspetto in modo flessibile attraverso tre slot di preferenza ordinati:

- **1ª preferenza** – il giorno libero ideale del docente (es. “sabato”).
- **2ª preferenza** – il giorno di ripiego se il primo non è possibile (es. “mercoledì”).
- **3ª preferenza** – l’ultima alternativa (es. “lunedì”).

Per ciascuna preferenza il docente (o il coordinatore) sceglie se è **HARD** (assoluta) o **SOFT** con un peso. Più un campo `required_free_days_count` indica *quanti* giorni liberi servono in totale (di default 1 per CCNL).

Esempio: un docente con preferenze [sabato **HARD**, mercoledì **SOFT** peso 50] e `count=1` vorrà sicuramente sabato libero, e in secondo ordine il mercoledì se il sabato non è fattibile.

3.8 Preferenze di giorno libero della classe

Anche le classi possono avere giorni preferiti di non-attività. La struttura è analoga a quella dei docenti: tre slot ordinati con flag HARD/SOFT, più un **required_free_days_count**.

In aggiunta le classi hanno il campo **max_hours_per_day** (default 5) che impone un tetto HARD al numero di ore in un giorno. Riducendolo (a 4 per esempio) si ottengono giornate più leggere ma se il monte ore della classe è alto bisognerà estendere il calendario settimanale al sabato.

4. Workflow di ottimizzazione

4.1 Schema delle fasi

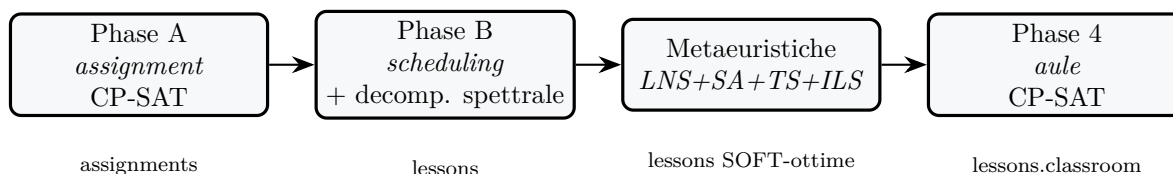


Figura 4.1: Pipeline a 4 fasi.

4.2 Phase A: assegnazione (cattedre)

Modulo. `experiments/cpsat_v2_assignment.py`.

Lancio. `POST /api/optimize/assignment` con `{time_limit_s, workers, log}`.

Input. L'insieme di `(class, subject, hours_per_week)` letto da `class_subjects` e/o `curriculum_subjects` + le abilitazioni docente in `teacher_subjects` + la classe-di-concorso preference in `subject_group_weights`.

Output. La tabella `assignments` (`teacher x class x subject` con `hours`). Problema di covering: ogni `(class, subject)` riceve un docente abilitato, rispettando `max-hours` per docente, classi-di-concorso e `teacher_compatible_classes`.

4.3 Phase B: scheduling

Modulo. `experiments/cpsat_v2_timetable.py` + `experiments/decomposition_spectral_v2.py`.

Decomposizione spettrale. Per istanze grandi, il problema CP-SAT integrale è intrattabile. La pipeline calcola un grafo di co-occupazione fra docenti, ne fa un'embedding spettrale via Laplaciano normalizzato e usa K-means su quell'embedding per partizionare in k cluster *loosely-coupled*. Ogni cluster diventa una sub-istanza CP-SAT indipendente; un secondo step "bridges" risolve solo i bordi; un terzo step "ricucitura" fa full-CPSAT con tutto il problema partendo dalla soluzione stitched.

4.4 Metaeuristiche

Modulo. `experiments/metaheuristics.py`. Cascata di:

- **LNS** – destroy-and-repair su finestre (`teacher, day`) o (`class, day`); freeze del 60-70% delle lezioni e CP-SAT-resolve del resto.
- **SA** (Simulated Annealing) – single-swap accettati con Metropolis a T decrescente con fattore α .
- **TS** (Tabu Search) – best-improvement local search con tabu list (default size 80).
- **ILS** (Iterated Local Search) – alterna n cicli di TS con *kick* perturbativi (random 4-opt fra giorni).

Tutti contribuiscono al SOFT score: holes per teacher, isolated 1- and 5-hour days, weekly distribution, dual-hour pairs, no 6th hour, banded preferences su materie. Il SOFT include la contribuzione delle matrici di disponibilità SOFT/PREFERITO e dei logical SOFT/PREFERITO.

4.5 Phase 4: assegnazione aule

Modulo. `experiments/classroom_assignment.py`. Input: la tabella `lessons` (già temporalmente fissata) + le classroom rules (`kind, capacity, multi_class`, preferenze materia↔aula, classe↔aula, docente↔aula). Output: `lessons.classroom_name` popolato.

4.6 Drag-and-drop con preview live

Trascinando una lezione su un altro slot la UI fa due chiamate:

1. POST `/api/schedule/move-preview` – il backend simula la mossa per tutte le 36 (`day, hour`) e per ognuna restituisce:
 - `ok` con `delta_soft <= 0` – mossa accettabile (verde se `delta < 0`, verde chiaro se `delta = 0`).
 - `soft_worse` – mossa accettabile ma peggiora il SOFT (giallo).
 - `hard_violation` – vincolo HARD violato (rosso).
 - `noop` – slot di origine.
2. L'utente droppa, la UI invia PUT `/api/schedule/move-lesson` che valida e persiste.

4.6.1 Room-follows-lesson

Il move-preview NON considera l'aula nel calcolo HARD. Se la nuova posizione è compatibile con docente / classe ma l'aula attuale è già occupata o HARD-unavailable in destination, il backend:

1. Sposta la lezione.
2. Lascia l'aula libera sul nuovo slot solo se non c'è conflitto.
3. Se c'è conflitto, rimuove l'aula dalla lezione spostata e risponde con `room_cleared=true, cleared_room=<nome>`.
4. La UI apre un Modal che spiega cosa è successo e chiede di scegliere una nuova aula dal dropdown (verde libera / rosso occupata).

4.7 Slot picker matriciale (Monitor)

Nel tab *Monitor*, espandendo una cattedra si vedono le singole lezioni; cliccando il bottone *Giorno/Ora* si apre un modal con una matrice 6x6 dove ogni cella è colorata in base alla disponibilità:

- verde: free (docente e classe liberi)
- rosso: HARD (docente o classe già impegnati)
- ambra: aula occupata da un'altra lezione
- azzurro: slot attuale

Click su una cella libera fa un dry-run via PUT `/api/monitor/event/{aid}/lesson/{lid}`. Se ci sono conflitti, apre un secondo Modal con 3 opzioni: **Annulla**, **Disassegna le lezioni in conflitto**, **Disassegna e ottimizza dopo**.

4.8 Punteggio di graduatoria e criterio Anzianita

Ciascun docente nell'anagrafica ha un campo **graduatoria_score** (numerico). Il valore esprime il punteggio di graduatoria utile a discriminare i docenti quando si tratta di assegnare cattedre o supplenze: più alto = maggiore precedenza.

Il campo è usato in due punti:

1. **Modulo supplenze**: quando il sistema cerca un candidato per coprire un'assenza, ordina la rosa per **graduatoria_score** decrescente.
2. **Phase A, criterio “Anzianita”**: il DSL della funzione obiettivo della Phase A include un preset **Anzianita** che, oltre ai criteri standard di bilancio (capacità non utilizzata, n. di indirizzi, ecc.), privilegia le assegnazioni che fanno coincidere docenti con alto **graduatoria_score** con classi del triennio finale (a parità di altre condizioni).

Il preset è selezionabile dalla card **Phase A (assegnazione)** del tab Workflow, dropdown “Criterio”. Il DSL completo della funzione obiettivo è visibile nel modal “Mostra DSL attivo” della stessa card.

5. Guida UI

5.1 Funzionalità trasversali

5.1.1 Query DSL e sort multi-livello

Ogni pagina lista ha:

- barra Query (DSL: =, !=, <, <=, >, >=, contains, startswith, endswith, in [...], logica AND / OR, parentesi);
- sort multi-livello (max 4 livelli);
- pannello Help con la lista dei campi.

5.1.2 Multi-select

Sulle pagine Docenti / Classi / Aule la lista supporta:

- click semplice: single-select (replace);
- Ctrl+click: toggle;
- Shift+click: range;
- bottone "Vincolo collettivo" che apre il *BulkApplyModal*.

5.1.3 Excel/CSV import

Ogni pagina lista ha "Importa Excel/CSV" + "Template". Lo stesso endpoint POST `/api/import/{entity}` gestisce le 7 entità (teachers, subjects, classes, classrooms, curricula, students, groups). Modalità: **upsert** (default), **append**, **replace**. Header alias italiani / inglesi accettati.

5.1.4 Esportazione liste (xlsx / csv)

Sopra ogni tabella, accanto a "Reset query" / "Reset sort", trovi 3 bottoni di esportazione:

- **xlsx**: la *vista corrente* (solo le righe filtrate dalla query DSL, nell'ordine determinato dal sort multi-livello) come file `.xlsx` con header colorato e auto-fit colonne.
- **csv**: stesso contenuto, formato CSV UTF-8 con BOM (compatibile con Excel italiano).
- **tutto**: ignora query e sort, esporta tutte le righe dell'entità.

Filename: `<entita>_<YYYYMMDD>_<HHMMSS>.<ext>`. I parametri `?format=xlsx|csv` sono accettati da tutti gli endpoint di lista, quindi gli stessi export sono disponibili anche via curl.

5.1.5 Viste salvate

Sopra ogni tabella, accanto ai bottoni di Reset e Export, trovi un dropdown "Viste salvate..." e i bottoni "Salva vista" / "x":

- **Salva vista:** chiede un nome e memorizza la query DSL + il sort multi-livello come `SavedView(entity, name, dsl_query, sort_levels)`. La vista è globale.
- **Dropdown "Viste salvate...":** applica una vista (sostituisce query + sort + ricarica la lista).
- **x:** elimina la vista correntemente selezionata.

Endpoint: `/api/saved-views?entity=<entity>` (GET / POST / PUT {id} / DELETE {id}). Tabella: `saved_views`.

5.1.6 Tag (aule e studenti)

Sistema many-to-many parallelo per aule e studenti.

Tag aule: tabella `classroom_tags` + join `classroom_tag_assignments`. UI: multi-select autocomplete nella scheda aula, bottone "Gestisci tag" per CRUD globale, colonna "Tag" con pill colorate (un colore stabile per nome). Il generatore mock tagga automaticamente le aule per kind (`lab_fisica` → [`lab`, `fisica`, `scienze`], `palestra` → [`palestra`, `motorie`], ecc.) e propaga il curriculum dalla home class (`3B_scientifico` → tag `scientifico`).

DSL: `"matematica"` in `l.classroom.tags` (vincoli) o `has_tag(matematica)` (query lista). I tag sconosciuti vengono creati al volo al salvataggio.

Tag studenti: tabella `student_tags` + join `student_tag_assignments`. Casi d'uso: BES, DSA, `debito_matematica_4`, `pcto_ditta_<x>`, `studente_atleta`. UI analoga a quella delle aule. Pannello "Precompila da tag" nel modal *Gruppi* che aggiunge in massa gli studenti che matchano `any_of` o `all_of` su una lista di tag.

Importante: i tag NON sostituiscono i gruppi. I gruppi restano l'unità operativa di scheduling, i tag sono metadato passivo per filtrare/raggruppare.

5.1.7 Import bulk dedicato (/import)

Pagina dedicata raggiungibile dalla nav. Layout a due colonne: configurazione (entità, modalità, "Scarica template") + file da importare (drop area + selettore file). Il template `xlsx` ha 3 fogli:

- **Istruzioni:** tabella **Campo** | Hint con descrizione di ogni colonna + note generali (sinonimi italiani, righe vuote ignorate, etc.);
- **Esempi:** una riga di esempio pre-popolata;
- **Dati:** foglio vuoto da compilare. L'importer legge per default questo foglio.

Campi importabili già integrati con le feature recenti: `tags` su `students` e `classrooms`, `graduatoria_score` / `required_free_days_count` / `preferred_free_days_json` su `teachers`, e `max_hours_per_day` su `classes`.

5.1.8 Import / Export DB (Dashboard)

Card prominente nella prima riga della Dashboard. Funzioni:

- **Esporta DB completo:** scarica un `.zip` contenente `database.db` (raw SQLite), `tables/<t>.csv` per ogni tabella e `metadata.json` (schema_version + sha256).

- **Solo schema:** omette i dati di scheduling – utile come template per inizializzare un'altra scuola.
- **Import:** selettore file .zip + conferma esplicita. Una copia `timetable.db.pre_import_backup` viene salvata prima dello swap.
- **Snapshot:** timestampati in `webui/data/snapshots/`, lista con bottoni Ripristina / Elimina.

Endpoint: `/api/dashboard/export-db`, `/api/dashboard/import-db`, `/api/dashboard/snapshot/{create|update|DELETE /api/dashboard/snapshot/{f}}`.

5.2 Tab per tab

Dashboard

Importa profilo (5 size) con 3 checkbox per i pool (curricula, aule, studenti); genera scuola di test; stato corrente con 7 card-numero; *RunLogPanel* streaming.

Docenti, Classi, Indirizzi, Studenti, Gruppi, Materie, Aule, Compresenze

CRUD entità con modal di edit. Vedere `ui_guide.md` per i campi specifici di ognuna.

Cattedre

Layout a card: una card per classe, lista materie+docente+ore. Bottone "cambia": apre modal con dropdown filtrato per materia, ogni opzione mostra `Cognome Nome - assigned/max [SFORA: +Xh] / [pieno]`. Bottone "Warnings cattedre": pannello con docenti SFORA / sotto / vuoti / ok.

Orario

4 viste (per classe, per docente, per aula, per slot), ognuna con toggle matrice/lista. Drag-and-drop con preview live, dropdown aule colorate (verde libera / rosso occupata).

Assenze e supplenze

Vista settimanale 6×6. Click su intestazione giorno: modal gestione assenze. Click su cella rossa: modal con classi scoperte (sinistra) + docenti disponibili (destra), drag-drop per assegnare supplenti.

Monitor

Lista *eventi* (uno per Assignment). Espansione lezione-per-lezione con bottone Giorno/Ora che apre slot picker matriciale (vedere §5.6). In cima alla pagina un *segmented control* con quattro tab per filtrare velocemente: **Tutti**, **Incompleti** (eventi con `n_assigned_hours < n_expected_hours`), **Senza orario** (eventi senza alcuna lezione piazzata) e **Lockati** (eventi con almeno una lezione marcata `lesson.locked == true`). Per ogni riga, le azioni disponibili sono *Dissocia*, *Blocca/Sblocca* e *Piazza* (eseguito singolarmente o in massa via multi-selezione + toolbar bulk).

Vincoli

Lista flat di tutti i vincoli editabili con pill colorate per livello. Bottone "Cerca conflitti": pannello con conflitti rilevati.

Workflow

Pulsanti per lanciare le 4 fasi separatamente o "Pipeline completa". SSE log streaming.

6. Launcher

6.1 Windows: `start.bat`

Apri due finestre cmd separate. Pre-flight:

- `npm` reachable (forza `nodejs` nel PATH);
- `venv` esiste in `backend/.venv`;
- `backend/main.py` e `frontend/package.json` presenti;
- se `node_modules` manca, lancia `npm install`;
- avvisa se 8000 / 5173 sono già occupate.

I path sono relativi a `%~dp0`, quindi lo script funziona ovunque venga messo.

6.2 Linux / macOS: `start.sh` + `stop.sh`

Default background: log in `webui/logs/`, PID in `webui/logs/pids`. Flag `-f` / `-foreground` per Ctrl+C cleanup. Auto-crea `venv` e fa `npm install` se mancano. Hint per `apt`, `dnf`, `pacman`, `brew` se `python3` / `npm` non sono nel PATH.

`stop.sh` legge `logs/pids`, manda SIGTERM con fallback SIGKILL dopo 3s, killa anche i child via `pkill -P` e ha un fallback per ammazzare qualunque processo ancora in ascolto su 8000 / 5173 via `lsof`.

Parte II

Appendici tecniche

7. Architettura

7.1 Stack

- **Backend:** Python 3.11+, FastAPI, SQLAlchemy 2.0, Pydantic 2, uvicorn, OR-Tools, NumPy, scikit-learn, Faker.
- **Frontend:** SvelteKit (Svelte 4), Vite 5, Tailwind CSS, JavaScript puro.
- **DB:** SQLite via SQLAlchemy. Migrazioni idempotenti nel codice (no Alembic).
- **Engine I/O:** pickle Python; `engine_io.py` converte fra DB e dict per il solver.

7.2 Diagramma a blocchi

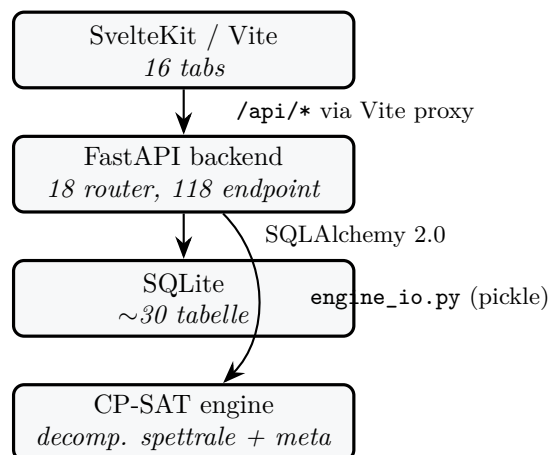


Figura 7.1: Architettura a blocchi della webapp.

7.3 Struttura cartelle

```
timetable/  
+-- README.md -- landing page  
+-- docs/ -- manuale tecnico  
+-- experiments/ -- solver code + pickle snapshots  
| +-- cpsat_v2_assignment.py -- Phase A  
| +-- cpsat_v2_timetable.py -- Phase B  
| +-- decomposition_spectral_v2.py  
| +-- metaheuristics.py -- LNS / SA / TS / ILS  
| +-- classroom_assignment.py -- Phase 4  
+-- schedule/ -- legacy single-file prototypes  
+-- webui/  
| +-- start.bat -- launcher Windows
```

```
| +-- start.sh -- launcher Linux/macOS
| +-- stop.sh -- stop helper
| +-- backend/ -- FastAPI app
| +-- frontend/ -- SvelteKit
| +-- data/timetable.db -- SQLite
| +-- docs/ -- guide brevi user-facing
+-- proposals/ -- design notes + benchmarks
+-- *.pdf -- reference papers
```

7.4 Lifecycle del backend

webui/backend/main.py definisce un lifespan che chiama `init_db()` allo startup. Esso fa:

1. `Base.metadata.create_all(bind=engine)` – crea le tabelle mancanti.
2. `_apply_lightweight_migrations()` – ALTER TABLE idempotenti per i campi aggiunti in versioni successive (esempio: `school_classes.curriculum_id`, `teachers.last_name/first_name/nickname` ecc.). Ogni ALTER è guardato da `PRAGMA table_info` per non duplicare.

8. Modello dati

Tutto vive in `webui/backend/models.py`. Le tabelle si raggruppano in cinque famiglie tematiche.

8.1 Famiglie di tabelle

Anagrafiche

`subjects`, `teachers`, `school_classes`, `classrooms`, `curricula`, `students`, `study_groups`.

Tag (M-N)

`classroom_tags` + `classroom_tag_assignments`, `student_tags` + `student_tag_assignments`.
Nome unique lower-cased, descrizione opzionale, cancellazione cascata.

Assegnazione

`class_subjects`, `curriculum_subject_hours`, `teacher_subjects`, `subject_group_weights`,
`assignments`.

Disponibilità / vincoli

`teacher_unavailability`, `class_unavailability`, `classroom_unavailability`, `logical_unavailabilitie`,
`curriculum_logical_constraints`, `classroom_subject_preferences`, `teacher_classroom_preferences`,
`coteaching_rules`.

Soluzioni e scheduling

`solutions`, `lessons`, `day_counts`.

Coverage e Run

`absences`, `substitute_assignments`, `runs`, `run_logs`, `app_state`.

Viste salvate

`saved_views`: query DSL + sort multi-livello salvati per una entità lista, esposti dal dropdown
"Viste salvate" del componente *SortableQueryableList*. UNIQUE (`entity`, `name`).

8.2 Diagramma relazioni

8.3 Migrazioni idempotenti

Pattern (in `db.py`):

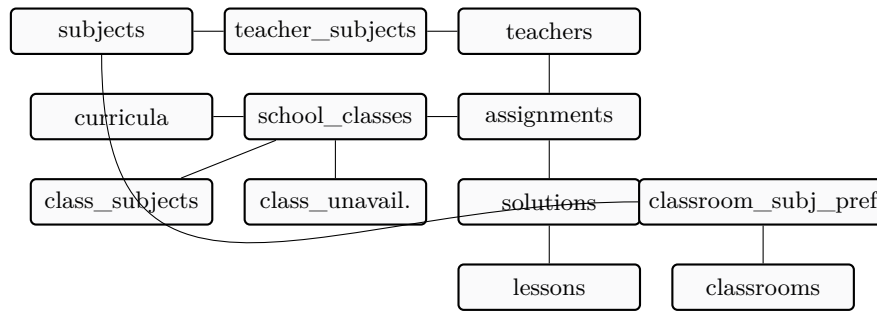


Figura 8.1: Relazioni rilevanti del modello dati (semplificato).

```

if insp.has_table("school_classes") \
    and not has_column("school_classes", "curriculum_id"):
    conn.execute(text(
        "ALTER TABLE school_classes ADD COLUMN curriculum_id INTEGER"
    ))

```

Ogni ALTER è guardato da una check su PRAGMA table_info. Quando aggiungi una colonna nuova, segui lo stesso pattern: il DB preesistente sopravvive senza interventi manuali.

8.4 Pickle dell'engine

engine_io.py converte fra DB e tre forme pickle:

```

school = {
    'profile': str,
    'classes': [{name, year, section, curriculum, subjects:{subj:ore}}],
    'teachers': [{name, group, max_hours, free_day,
                  weights:{subj:int}}],
    'cconcorsopersubject': {subj: {group: weight}},
    'curriculum_scores': {curriculum: int},
    'curricula': [...],
    'students': [...],
    'groups': [...],
}
profs = {
    teacher_name: {
        'classi': {class_name: {subject: {'ore': N}}},
        'glibero': [d1, d2, d3]
    }
}
solution = {(prof, class, subj, day, hour): 0|1}

```

9. DSL generico per i vincoli

Linguaggio compatto, dichiarativo, totalmente componibile per esprimere vincoli HARD/SOFT/PREFERITO/ENFORCED su qualunque combinazione logica di predicati sul modello dei dati. Implementato in `webui/backend/utils/general_dsl.py` (parser ricorsivo discendente + AST + evaluator post-hoc, ~250 LOC, no dipendenze esterne). Esposto via `POST /api/constraints/general`, validato live da `POST /api/constraints/general/validate`, accessibile dal modal “+ Nuovo vincolo DSL” del tab Vincoli.

9.1 Filosofia

Prima del DSL generico il sistema aveva quattro sub-DSL specializzati (query, logical, objective, introspettivo). Stessa grammatica implementata 4 volte. Il DSL generico unifica tutto sotto una sola grammatica con quattro “sapori” di compilazione (LIST, LOGIC, OBJECTIVE, EVAL): *un parser, molti compilatori*. La sintassi di `forall x in S where Filter: Body` e dei predicati atomici (`==`, `!=`, `<`, `in`, ...) è identica in tutti i contesti; cambia solo il backend di traduzione.

Il DSL è interpretato *post-hoc*: parsea l’espressione e la valuta DOPO che la pipeline (Phase A, Phase B, metaeuristiche) ha prodotto una soluzione candidata. Le violazioni HARD sono rilevate dal Feasibility Check; le SOFT alimentano lo score globale.

9.2 Grammatica BNF

```
expr ::= iff_expr
iff_expr ::= implies_expr ( "<=>" implies_expr ) *
implies_expr ::= or_expr ( "=>" or_expr ) *
or_expr ::= and_expr ( ("OR"|"or") and_expr ) *
and_expr ::= not_expr ( ("AND"|"and") not_expr ) *
not_expr ::= ("NOT"|"not") not_expr | atom

atom ::= quant_expr | count_expr | comparison
      | call | bool_lit | "(" expr ")"

quant_expr ::= ("forall"|"exists") IDENT "in" source
            ["where" or_expr] ":" expr

count_expr ::= "count" IDENT "in" source ["where" or_expr]
            ( ":" comparison | op value )

source ::= IDENT -- literal name
        | IDENT ( "." IDENT ) + -- path (runtime)

comparison ::= value ( op value
                    | "in" "[" value ( "," value ) * "]"
                    | "in" path
                    | "not_in" "[" value ( "," value ) * "]"
```

```

        | "not_in" path )

op ::= "==" | "!=" | "<" | "<=" | ">" | ">="
value ::= IDENT ( "." IDENT )* | NUM | STRING
        | BOOL | function_call
function_call ::= IDENT "(" [arg_list] ")"
arg_list ::= arg ("," arg)*
arg ::= IDENT "==" value | value

```

Precedenze (dal più basso al più alto): <=>, ==>, OR, AND, NOT, confronti, . (member access). Le parentesi sovrascrivono.

9.3 Sorgenti iterabili

Sorgente	Tipo elemento
lessons	lezioni della soluzione attiva
assignments	cattedre (Assignment rows)
teachers	docenti (name, group, max_hours, subject)
classes	classi (name, year, curriculum, n_students)
classrooms	aule (name, kind, type, tags)
students	studenti (con tags[] e groups[])
subjects	materie
curricula	indirizzi di studio
groups	gruppi articolati
slots	le 36 (day, hour) della settimana
days	i 6 giorni (Lun..Sab; index 1..6)
hours	le 6 ore (8..13)

Sorgente-path: dalla v0.5 puoi scrivere `exists g in s.groups: g.name == "X"` dove `s.groups` e' un attributo che restituisce una lista; il parser accetta qualunque path puntato dopo `in`.

9.4 Galleria di esempi

Ogni esempio sotto e' stato validato live contro il parser corrente; copia-incolla nell'editor del modal "+ Nuovo vincolo DSL" e funziona.

9.4.1 Vincoli docente (10 esempi)

1. Borghi non insegna mai in 1A alla 6a ora:

```

forall l in lessons where l.teacher == Borghi
    and l.class == 1A:
    l.hour != 13

```

2. Borghi: max 4 ore/giorno totali:

```

forall d in days:
    count l in lessons where l.teacher == Borghi
        and l.day == d.index: l <= 4

```

3. Lab fisica: ogni docente di Fisica esattamente 1 ora a settimana in lab_fisica:

```

forall t in teachers where t.subject == "Fisica":
    count l in lessons where l.teacher == t
        and l.classroom.type == "lab_fisica": l == 1

```

4. Tetto cattedra Inglese: max 18h/settimana:

```
forall t in teachers where t.subject == "Inglese":
    count l in lessons where l.teacher == t: l <= 18
```

5. Rossi non lavora il sabato (HARD):

```
forall l in lessons where l.teacher == Rossi
    and l.day == 6:
    false
```

6. Rossi: al più 1 sesta ora/settimana:

```
count l in lessons where l.teacher == Rossi
    and l.hour == 13: l <= 1
```

7. Tutti A050: max 5 ore/giorno:

```
forall t in teachers where t.group == "A050":
    forall d in days:
        count l in lessons where l.teacher == t
            and l.day == d.index: l <= 5
```

8. Coupling Rossi 3A $\Rightarrow$ Bianchi 3B:

```
(exists l in lessons:
    l.teacher == Rossi and l.class == 3A)
=> (exists l in lessons:
    l.teacher == Bianchi and l.class == 3B)
```

9. Rossi mai in palestra:

```
forall l in lessons where l.teacher == Rossi:
    l.classroom.type != "palestra"
```

10. Tutti i prof di mate: almeno una lezione di mattina:

```
forall t in teachers where t.subject == "Matematica":
    exists l in lessons where l.teacher == t:
        l.hour <= 9
```

9.4.2 Vincoli classe (5 esempi)

1. La 1A non lavora il sabato:

```
forall l in lessons where l.class == 1A: l.day != 6
```

2. Le prime: max 5 ore/giorno:

```
forall c in classes where c.year == 1:
    forall d in days:
        count l in lessons where l.class == c.name
            and l.day == d.index: l <= 5
```

3. Quarte/quinte: due ore CONSECUTIVE di mate:

```
forall c in classes where c.year >= 4:
  exists s1 in slots: exists s2 in slots:
    consecutive(s1, s2)
    and lesson(class==c.name, day==s1.day,
               hour==s1.hour, subject==Matematica)
    and lesson(class==c.name, day==s2.day,
               hour==s2.hour, subject==Matematica)
```

4. Linguistico anno 1: niente sabato:

```
forall l in lessons
  where l.class.curriculum == Linguistico
  and l.day == 6: false
```

5. 1A: religione solo nelle prime 4 ore:

```
forall l in lessons where l.class == 1A
  and l.subject == Religione:
  l.hour <= 11
```

9.4.3 Vincoli aula (5 esempi)

1. Lab Fisica 1: una sola lezione per slot:

```
forall s in slots:
  count l in lessons
    where l.classroom == "Lab Fisica 1"
    and l.day == s.day and l.hour == s.hour:
  l <= 1
```

2. Matematica solo in aule taggate “matematica”:

```
forall l in lessons where l.subject == Matematica:
  "matematica" in l.classroom.tags
```

3. Storia delle terze: aula con proiettore:

```
forall l in lessons where l.subject == Storia
  and l.class.year >= 3:
  "proiettore" in l.classroom.tags
```

4. La palestra ospita SOLO Scienze motorie:

```
forall l in lessons
  where l.classroom.type == "palestra":
  l.subject == Scienzemotorie
```

5. Lab chimica: max 2 lezioni concorrenti:

```
forall s in slots:
  count l in lessons
    where l.classroom.type == "lab_chimica"
    and l.day == s.day and l.hour == s.hour:
  l <= 2
```


9.4.4 Vincoli materia (3 esempi)

1. Mate distribuita: max 2 ore/giorno per classe:

```
forall c in classes:
  forall d in days:
    count l in lessons
      where l.class == c.name
        and l.subject == Matematica
        and l.day == d.index: l <= 2
```

2. Mate preferibilmente entro la 5a ora:

```
forall l in lessons where l.subject == Matematica:
  l.hour <= 12
```

3. Educazione fisica solo in palestra:

```
forall l in lessons
  where l.subject == Educazione fisica:
    l.classroom.type == "palestra"
```

9.4.5 Vincoli gruppo / studenti (3 esempi)

1. Studenti BES devono essere in un gruppo Sostegno (uso di `exists g in s.groups`):

```
forall s in students where "BES" in s.tags:
  exists g in s.groups: g.name == "Sostegno"
```

2. Studenti con debito mate quarto → gruppo Recupero:

```
forall s in students
  where "debito_matematica_4" in s.tags:
    exists g in s.groups:
      g.name == "Recupero Matematica"
```

3. Studenti BES: nessuna lezione il sabato:

```
forall l in lessons:
  forall s in students
    where l.class == s.class
      and "BES" in s.tags:
        l.day != 6
```

9.4.6 Vincoli relazionali / coupling (4 esempi)

1. Rossi in Aula 12: solo Matematica:

```
forall l in lessons where l.teacher == Rossi
  and l.classroom == "Aula 12":
  l.subject == Matematica
```

2. Se Rossi in 3A allora deve usare lab fisica almeno una volta:

```
(exists l in lessons:
  l.teacher == Rossi and l.class == 3A)
=>
(exists l2 in lessons:
  l2.teacher == Rossi and l2.class == 3A
  and l2.classroom.type == "lab_fisica")
```

3. Inglese alle prime: max 3 ore/sett per (docente,classe):

```
forall t in teachers where t.subject == "Inglese":
    forall c in classes where c.year == 1:
        count l in lessons
            where l.teacher == t
                and l.class == c.name: l <= 3
```

4. Religione alle prime: mai in 1a ora:

```
forall l in lessons where l.subject == Religione
                        and l.class.year == 1:
    l.hour != 8
```

9.5 Errori comuni

- **Atteso COLON, trovato OP (=)** – usa == nei confronti dopo **where** (= singolo e' lecito, ma confonde il parser quando appare in contesti ambigui).
- **sorgente sconosciuta 'foo'** – sorgente non in `_VALID_SOURCES` e non e' un path raggiungibile via env. Controlla l'ortografia.
- **oltre 1000000 atomi: probabile esplosione combinatoria** – quantificatori troppo annidati. Sposta filtri nel **where** più esterno per ridurre il set; oppure splitta il vincolo in più vincoli salvati separatamente.

Vedere `docs/general_dsl.md` per la guida completa con sezioni 1–12 (galleria di 30+ esempi commentati, troubleshooting esteso, reference tabellare di tutti gli attributi per ciascuna entità).

10. Tecniche di ottimizzazione avanzate

Oltre alle metaeuristiche classiche (LNS / SA / TS / ILS) il progetto include cinque tecniche aggiuntive. Vedere `docs/optimization_strategies.md` per la specifica completa.

10.1 Adaptive Large Neighborhood Search (ALNS)

`experiments/alns.py`. Sei destroy operator (`random_window`, `day_cluster`, `worst_fit_day`, `teacher_day`, `classroom_day`, `single_class_week`) e tre repair operator (`cp_sat_window`, `greedy_by_soft`, `bfs_fill_back`) selezionati con roulette wheel sopra exponentially-decayed scores; acceptance SA-like.

10.2 Variable Neighborhood Search (VNS)

`experiments/vns.py`. Cicla quattro vicinati di dimensione crescente (1-swap, 2-swap, 3-chain, k-opt con $k \in [4, 6]$). Ad ogni miglioramento riparte dal vicinato 1; termina quando un'intera passata 1→4 non trova nulla.

10.3 Hall's theorem pre-check

`experiments/diagnostics/hall_check.py`. Diagnostico sincrono pre-Phase-A. Tre controlli:

1. per (class, subject) coverage,
2. subject supply vs demand,
3. Hall sampling random-subset con sottoinsiemi esclusivi.

Esposto in *tre* punti UI: bottone inline in PhaseACard, riga in AdvancedTechniquesCard, sezione del tab Statistiche.

10.4 Column Generation (Dantzig-Wolfe)

`experiments/column_generation.py`. Master LP via `scipy.linprog` HiGHS; sotto-problema per docente. Skeleton single-iteration per scuole molto grandi (> 200 classi). Default OFF nella pipeline.

10.5 Lagrangian Relaxation

`experiments/lagrangian.py`. Rilassa i ponti inter-cluster e li dualizza con multipliers λ_b ; aggiornamento via subgradient ascent $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$ con $\alpha_k = \alpha_0 / (1 + k)$. Per-iteration refinement via SA.

11. Diagnostica statistica

Il tab `/diagnostics` aggrega tutte le analisi sul modello e sulla soluzione attiva. Vedere `docs/diagnostics.md`. Visualizzazioni con Apache ECharts (lazy-loaded), tema *pitantum* (palette indaco / oro / avorio / terra-di-siena).

11.1 Sensitivity Monte Carlo

Genera N perturbazioni feasible (sequenze di mosse atomiche random accettate solo se HARD soddisfatto) e calcola statistiche SOFT. Coefficiente di variazione $< 0.05 \Rightarrow$ soluzione vicina all'ottimo locale.

11.2 Analisi bipartito

Tre metriche networkx: densita' del grafo classe $\times$ docente, modularita' Newman-Girvan greedy, top-K betweenness centrality.

11.3 Correlazioni e regressioni

Tre regressioni statsmodels: OLS `load_vs_gaps`, OLS `subjects_vs_soft`, Logit `saturation_vs_sixth`. Output con coefficienti, p-values, R^2 (o pseudo- R^2) e interpretazione testuale auto-generata.

11.4 Distribuzioni

Cinque distribuzioni (carichi docenti, classi per giorno, materia $\times$ slot heatmap, occupancy slot, KS-test, chi-quadro) + goodness-of-fit.

11.5 Run telemetry

Tabella `run_telemetry` (alembic `a848420325b3`). I moduli solver pushano sample via `utils.telemetry.collect` (phase). Il run detail page (`/runs/[id]`) mostra il grafico objective vs tempo (multi-linea per phase) + statistiche per stage + export JSON.

12. API REST

Tutti gli endpoint sono sotto `/api/`. Backend espone ~118 route. Lista non esaustiva delle più rilevanti.

12.1 Pattern CRUD

Ogni risorsa CRUD ha la stessa forma:

Verbo	Path	Descrizione
GET	<code>/api/<entity></code>	lista (q + sort)
GET	<code>/api/<entity>/<id></code>	dettaglio
POST	<code>/api/<entity></code>	create
PUT	<code>/api/<entity>/<id></code>	replace
DELETE	<code>/api/<entity>/<id></code>	delete

12.2 Esempi curl

```
# stato dataset
curl http://127.0.0.1:8000/api/dataset/state

# import small con tutti i pool
curl -X POST -H "Content-Type: application/json" \
  -d '{"profile":"small","use_optimized":true,
      "import_curricula":true,"import_classrooms":true,
      "import_students":true}' \
  http://127.0.0.1:8000/api/dataset/import-profile

# vincolo logico HARD su un docente
curl -X POST -H "Content-Type: application/json" \
  -d '{"expression":"mai aula:LabFisica@mar","kind":"hard"}' \
  http://127.0.0.1:8000/api/teachers/15/logical-unavailabilities

# bulk dry-run su 3 classi
curl -X POST -H "Content-Type: application/json" \
  -d '{"entity_ids":[1,2,3],"action":"add_logical",
      "payload":{"expression":"lun8 AND lun9","is_hard":true,
                  "soft_penalty":100},
      "on_conflict":"dry_run"}' \
  http://127.0.0.1:8000/api/bulk/classes/dry-run

# coverage settimana
curl 'http://127.0.0.1:8000/api/coverage/week?week_start=2026-04-27'

# event lessons (monitor)
curl http://127.0.0.1:8000/api/monitor/event/1/lessons
```

12.3 Riferimento gruppi di endpoint

- **Anagrafiche:** /api/teachers, /classes, /classrooms, /subjects, /curricula, /students, /groups.
- **Cattedre:** /api/assignments + teachers-for-subject + loads.
- **Vincoli logici:** /api/{teachers,classes,classrooms}/{id}/logical-unavailabilities, /api/curricula/{cid}/logical-constraints.
- **Bulk ops:** /api/bulk/{entity}/dry-run|apply.
- **Schedule:** /api/schedule/by-class|by-teacher|by-room|by-slot, move-lesson, move-preview, lesson/{lid}/classroom.
- **Coverage:** /api/coverage/week|cell, /api/absences, /api/substitutions.
- **Optimization:** /api/optimize/assignment|phase-b|lms|sa|ts|ils|full|classroom-assignment.
- **Monitor:** /api/monitor/events|summary|constraints|conflicts, event/{aid}/lessons|lesson/{id}.
- **Import:** /api/import/{entity}, /api/import/{entity}/template.

13. Estendere il sistema

13.1 Aggiungere una nuova tabella + CRUD

1. Modello in `webui/backend/models.py`.
2. Pydantic in `schemas.py` (`XxxIn`, `XxxOut`).
3. Router in `routers/xxx.py`. Copia un router esistente come template.
4. Includi il router in `main.py`: `app.include_router(xxx.router)`.
5. Adapter DSL in `utils/list_query.py`.
6. Frontend: nuova route `webui/frontend/src/routes/xxx/+page.svelte`, riusando *SortableQueryableList*.
7. Voce in `+layout.svelte` per la nav.

13.2 Migrazione idempotente

Aggiorna `db.py::_apply_lightweight_migrations`. Pattern:

```
if insp.has_table("my_table") \
    and not has_column("my_table", "new_field"):
    conn.execute(text(
        "ALTER TABLE my_table ADD COLUMN new_field VARCHAR(64)"
    ))
```

13.3 Aggiungere un nuovo tipo di vincolo

Per-cella. Copia il pattern di `TeacherUnavailability`: `unique (entity_id, day, hour)`, `columns state/penalty/reason`. Aggiorna `models.py`, `schemas.py`, router, `optimization.py::_availability_constraints`, `routers/monitor.py::_build_constraints` (per il tab Vincoli), `routers/bulk.py::ENTITY_UNAV_MODEL` se vuoi supportare bulk.

Logico. Aggiungi `entity_type` a `LogicalUnavailability`; aggiorna `routers/logical.py::ENTITY_MODEL` e `ENTITY_TYPE`; modifica `routers/monitor.py::_build_constraints`; aggiungi il branch nel solver.

13.4 Aggiungere un nuovo step di ottimizzazione

```
def my_new_step(...):
    params = dict(...)
    run_id = create_run("my_step", "Nome leggibile", profile, params)

    def target(rid: int):
        # carica dati, lancia solver, persiste
        update_run(rid, progress=0.5)
        ...
        update_run(rid, solution_id=sid, obj_value=v, metrics=m)
        update_run(rid, progress=1.0)

    start_thread(run_id, target)
    return run_id
```

Esponi via `routers/optimize.py` con un endpoint POST. Frontend in `/optimize` per il bottone di lancio + log streaming via *RunLogPanel* (SSE).

13.5 Testing

Non c'è una test suite formale. Per smoke-testare end-to-end:

- Backend boot pulito: `uvicorn backend.main:app`.
- Live curl: vedere §7.2.
- Frontend `vite build` deve passare clean.
- Migrazione idempotente: rilancia su DB esistente per confermare zero data loss.

A. Repository GitHub

URL: <https://github.com/gborghi/timetable>.

Repository privato. La landing page contiene il README di alto livello con quickstart, tour della UI, sintassi vincoli, architettura, repository layout.

B. Glossario discorsivo dei metodi

Questo capitolo spiega *cos'è* ogni metodo che π Tantum usa internamente, con un linguaggio che si possa leggere “dal divano”. Per ogni metodo trovi quattro cose in sequenza: l’idea, come funziona, quando usarlo, un esempio legato al timetabling. Niente formule a meno che non aggiungano chiarezza.

B.1 CP-SAT (Constraint Programming over SAT)

Cos'è e perché esiste. CP-SAT è il motore con cui π Tantum trova le soluzioni. È un *constraint solver*: gli dai un mucchio di variabili (per esempio: “in quale slot va la lezione di mate della 1A?”), un mucchio di vincoli (“due lezioni della stessa classe non possono stare nello stesso slot”; “il prof Borghi non insegna sabato”), e gli chiedi di trovare un assegnamento di valori che li rispetti tutti, possibilmente ottimizzando un criterio.

A differenza dei solver *lineari* (LP/MIP) che lavorano su disuguaglianze numeriche, CP-SAT lavora su variabili booleane e intere, e sa esprimere vincoli “logici” complessi (disgiunzioni, implicazioni, conteggi) in modo nativo. Questo lo rende particolarmente adatto al timetabling, dove la maggior parte dei vincoli sono di natura combinatoria, non geometrica.

Come funziona. CP-SAT alterna due fasi:

- **Propagazione:** data una scelta parziale (es. “ho deciso che mate-3A va in lunedì 8^a”), il solver *deduce* automaticamente quello che ne consegue (“allora ita-3A non può andare in lunedì 8^a”; “allora il prof Borghi è occupato in quello slot”, ecc.). Questo restringe lo spazio delle scelte rimanenti senza dover provare tutte le combinazioni.
- **Ricerca:** quando la propagazione esaurisce le conseguenze automatiche, il solver fa un *salto d'azzardo* (es. “provo a mettere ita-3B in martedì 9”) e si rimette a propagare. Se a un certo punto trova una contraddizione, fa *backtracking*: torna indietro, marca quella scelta come “no”, e prova un’alternativa. Impara dai fallimenti aggiungendo *clausole imparate* che evitano di ripetere lo stesso errore.

Quando usarlo. CP-SAT è il default di π Tantum per le Phase A e B. Per scuole di dimensioni normali (fino a ~80 classi) basta da solo. Per istanze più grandi serve combinarlo con la decomposizione spettrale o con metaeuristiche.

Esempio. Tipica iterazione: dopo aver propagato per qualche secondo, il solver ha già piazzato il 70% delle lezioni del triennio (perché sono molto vincolate dai lab e dalle compresenze) ed esplora le combinazioni rimanenti per le classi del biennio (più flessibili). Trova una contraddizione (un docente avrebbe due classi nello stesso slot), *backtracking*, ne prova un’altra. Tipicamente in 1-3 minuti chiude su una soluzione feasible.

B.2 Decomposizione spettrale

Cos'è e perché esiste. Quando la scuola è grande (>120 classi), il problema diventa troppo grosso perché CP-SAT lo digerisca in tempi ragionevoli. La decomposizione spettrale risolve

questo dividendo la scuola in piccoli “cluster” di classi che condividono pochi docenti. Ogni cluster diventa un sotto-problema piccolo, risolvibile in parallelo.

L’idea è: i conflitti veri (due docenti che competono per lo stesso slot) avvengono fra classi che condividono lo stesso docente. Se due classi non hanno docenti in comune, si possono pianificare totalmente in parallelo.

Come funziona. Si costruisce il grafo delle classi: nodi = classi, peso dell’arco fra due classi = numero di docenti condivisi. Si calcolano gli autovettori del *Laplaciano* di quel grafo (una matrice che misura, per ogni nodo, quanto si “differenzia” dai vicini). Gli autovettori associati ai più piccoli autovalori diversi da zero indicano *direzioni di separazione* naturali: progettando i nodi su quelle direzioni, si vedono chiaramente i cluster.

Un colpo di k-means su questa proiezione restituisce la partizione finale.

Quando usarlo. Default ON in π Tantum quando la scuola ha ≥ 120 classi. Per scuole più piccole non porta vantaggi e aggiunge overhead.

Esempio. Per una scuola di 250 classi, una decomposizione in 4 cluster produce sotto-problemi di 60-70 classi ciascuno, risolvibili in parallelo. I docenti-ponte (quelli che insegnano in classi di più cluster) sono il “vincolo di ricucitura” che la fase finale risolve come problema piccolo a se’ stante.

B.3 LNS (Large Neighborhood Search)

Cos’è e perché esiste. Quando hai già una soluzione (anche sub-ottima), spesso è più rapido *aggiustarla* che ricominciare. LNS prende la soluzione attuale, ne “distrugge” una porzione (es. tutte le lezioni di un certo giorno) e la ricostruisce con CP-SAT cercando un miglioramento. Iterando, esplora tante varianti vicine alla soluzione corrente.

Come funziona. A ogni iterazione: scegli un “vicinato” (es. day = martedì), libera tutte le variabili in quel vicinato, lancia CP-SAT con un piccolo budget di tempo per re-piazzarle ottimizzando lo SOFT score. Se il risultato è migliore, lo accetti.

Quando usarlo. Default ON nella pipeline integrata, dopo Phase B. Adatto al “polishing” di una soluzione già feasible.

Esempio. Dopo Phase B la soluzione ha 22 buchi nei docenti. LNS cicla: prova a re-fare lunedì (scende a 20 buchi), martedì (scende a 19), mercoledì (resta 19), ..., dopo qualche minuto si stabilizza intorno a 14-15.

B.4 ALNS (Adaptive LNS)

Cos’è e perché esiste. LNS classico ha un problema: usa sempre lo stesso tipo di vicinato (es. “un giorno”). Su alcune istanze è perfetto, su altre è sterile. ALNS lo evolve: invece di una sola scelta di vicinato, ne ha sei (un giorno intero, un cluster di classi, le ore peggiori della settimana, una giornata di un docente, una giornata di un’aula, una settimana di una sola classe) e tre modi di ricostruire (CP-SAT mini-window, greedy SOFT, BFS riempimento). A ogni iterazione sceglie quale combinazione usare con probabilità proporzionale a un *punteggio* che cresce per le combinazioni storicamente vincenti e decresce per le altre.

Come funziona. È un *roulette wheel selector* sopra punteggi esponenzialmente decadenti. Ogni operatore parte con punteggio neutro; ogni volta che produce un miglioramento, il punteggio sale; ogni volta che fallisce, scende. Aggiunge anche un’accettazione *simulated annealing*-like: occasionalmente accetta peggioramenti per sfuggire ai minimi locali.

Quando usarlo. Sostituisce o segue il LNS classico nella pipeline. Default ON.

Esempio. Su una scuola con cluster ben separati, l’ALNS impara dopo poche iterazioni che “cluster di classi” porta i miglioramenti più grossi e “giornata di un’aula” quasi mai funziona. Lo dice nei log finali con i punteggi.

B.5 SA (Simulated Annealing)

Cos'è e perché esiste. Il nome viene dalla metallurgia: come un metallo riscaldato e poi raffreddato lentamente forma una struttura cristallina più stabile, l'algoritmo accetta inizialmente anche peggioramenti per esplorare lo spazio delle soluzioni, e con il tempo (la “temperatura” che scende) diventa più selettivo, accettando solo miglioramenti.

Come funziona. A ogni iterazione genera una mossa random (es. scambia due lezioni). Se migliora lo SOFT, accetta; se peggiora di Δ , accetta con probabilità $e^{-\Delta/T}$ dove T è la temperatura corrente. La temperatura inizia alta (es. $T_0 = 10$) e si moltiplica per $\alpha < 1$ (es. $\alpha = 0.995$) a ogni iterazione, quindi decresce esponenzialmente.

Quando usarlo. È perfetto quando si sospetta di essere in un minimo locale. Default ON dopo LNS/ALNS.

Esempio. Dopo LNS ti sei stabilizzato a SOFT=420. Lanci SA con $T_0=10$, $\alpha=0.995$, budget 60s. Nei primi secondi, T è alto e accetta peggioramenti fino a 460-470; poi T scende e SA si concentra su miglioramenti, finendo per esempio a 405.

B.6 TS (Tabu Search)

Cos'è e perché esiste. Risolve un problema della ricerca locale: i loop. Se l'unica mossa migliorante è “ $A \rightarrow B$ ” e poi l'unica migliorante da B è “ $B \rightarrow A$ ”, sei intrappolato. TS evita questo tenendo una *lista tabu* delle ultime mosse fatte: per un certo numero di iterazioni, quelle mosse sono vietate. L'algoritmo è così forzato a esplorare zone diverse.

Come funziona. A ogni iterazione, valuta tutte le mosse possibili *tranne* quelle in tabu. Sceglie la migliore, la fa, aggiunge l'inversa alla lista tabu (con un “tempo di scadenza”). La lista è di dimensione fissa (parametro `tabu_size`, default 80).

Quando usarlo. Default ON dopo SA. Particolarmente efficace su istanze con plateau (zone dove molti vicini hanno lo stesso valore).

Esempio. Senza TS, scambiare due lezioni L_1 e L_2 produce SOFT=400; ri-scambiarle riproduce SOFT=405. SA potrebbe oscillare. TS dopo lo scambio mette “inverso di $L_1 \leftrightarrow L_2$ ” in tabu, quindi al passo successivo proverà $L_3 \leftrightarrow L_4$, esplorando una direzione nuova.

B.7 ILS (Iterated Local Search)

Cos'è e perché esiste. Combina ricerca locale con perturbazioni periodiche. Idea: dopo che la ricerca locale si è stabilizzata su un minimo locale, scuoti la soluzione (perturbazione: tante mosse random) e fai ricerca locale da capo. Iterando, esplori bacini di attrazione diversi.

Come funziona. Loop di “cycles” (default 3): in ognuno fai ricerca locale (TS) per un budget; se sei migliorato, salvi; poi perturba (rimuovi e ri-piazza un cluster random di classi); ricomincia.

Quando usarlo. Default ON come ultima fase prima del rooms assignment. È il *closer* della pipeline metaeuristica.

Esempio. Cycle 1 chiude su SOFT=400; perturba e ricomincia da SOFT=480; cycle 2 chiude su SOFT=395; perturba e ricomincia da SOFT=470; cycle 3 chiude su SOFT=392.

B.8 VNS (Variable Neighborhood Search)

Cos'è e perché esiste. VNS è una rifinitura sistematica. Cicla attraverso vicinati di dimensione crescente: prima prova 1-swap (scambia due lezioni), poi 2-swap (due paia), poi 3-chain (catena di 3 mosse), poi k-opt (catena di 4-6). Ogni volta che trova un miglioramento riparte dal vicinato 1; quando un intero ciclo passa senza miglioramenti, si ferma – significa che ha raggiunto un ottimo locale rispetto a tutti i vicinati.

Come funziona. For ogni vicinato, esplora fino a un budget di iterazioni; al primo miglioramento accetta e torna al vicinato 1. Quando il giro $1 \rightarrow 4$ produce zero miglioramenti, esce.

Quando usarlo. Come rifinitura finale, dopo LNS/ALNS/SA/TS. Default OFF (lo accendi quando vuoi spremere il massimo).

Esempio. Hai SOFT=395 dopo ILS. VNS cicla: 1-swap trova un miglioramento a 392; riparte. 1-swap trova ancora a 389; riparte. 1-swap zero; 2-swap trova a 385; riparte. Dopo qualche minuto chiude a 378.

B.9 Hall's theorem pre-check

Cos'è e perché esiste. Prima di lanciare un solver lungo, conviene controllare se il problema è *strutturalmente* risolubile. Il teorema di Hall (1935) dice: in un *matching bipartito* (assegnare ognuno di N elementi a uno di M elementi compatibili), una soluzione esiste sse per ogni sottoinsieme S degli N elementi, il loro vicinato (gli M compatibili con almeno uno) ha cardinalità $|N(S)| \geq |S|$.

Nel timetabling l'analogo pesato è: per ogni sottoinsieme di docenti, la loro capacità totale di ore deve coprire la domanda totale di ore delle materie che insegnano. Se una materia richiede 40 ore/settimana ma i docenti qualificati ne hanno solo 30 totali, il modello è già infeasible: non c'è bisogno di lanciare CP-SAT.

Come funziona. Tre controlli in cascata: per ogni materia controlla che esista almeno un docente qualificato; per ogni materia controlla che la capacità totale dei docenti qualificati copra la domanda; campiona N sottoinsiemi random di docenti e verifica che la domanda delle materie *esclusive* a ciascun sottoinsieme sia coperta dalla capacità del sottoinsieme.

Quando usarlo. Sempre prima di Phase A. Default ON nella pipeline integrata; se trova violazioni, interrompe immediatamente con un report.

Esempio. Hai aggiunto in mezzo all'anno una nuova materia "Diritto" al triennio (3 ore/settimana per 9 classi = 27 ore) ma hai un solo docente abilitato a Diritto con massimo 18 ore. Hall te lo dice in 100 millisecondi: "Diritto: domanda 27h > capacità docenti 18h". Risparmi 10 minuti di solver lungo.

B.10 Lagrangian Relaxation con sub-gradient

Cos'è e perché esiste. Quando un problema ha "vincoli scomodi" (vincoli che spezzano una struttura altrimenti separabile), una tecnica classica è *rilassarli*: li rimuovi dal modello e li reintroduci come penalità moltiplicate per coefficienti chiamati *moltiplicatori di Lagrange* (λ). Iterando, i λ si aggiustano per spingere la soluzione del modello rilassato a rispettare comunque i vincoli originari. È un modo di trasformare un problema "a blocchi accoppiati" in una sequenza di sotto-problemi indipendenti.

Nel timetabling questo è utile dopo la decomposizione spettrale: i cluster sarebbero indipendenti se non fosse per i *docenti-ponte* (docenti che insegnano in più cluster). Lagrangian relaxation rilassa la non-sovrapposizione dei docenti-ponte, e itera per riconciliare le scelte dei cluster.

Come funziona. Il *subgradient method* aggiorna λ a ogni iterazione: $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$, dove g_k è la violazione del vincolo rilassato e $\alpha_k = \alpha_0 / (1 + k)$ è una sequenza di "passi" che diminuisce a passi piccoli, garantendo convergenza al limite (in teoria) ma in pratica producendo soluzioni utili in poche iterazioni.

Quando usarlo. Scuole con cluster ben separati ma con docenti-ponte fastidiosi. Default OFF (avanzato).

Esempio. Cluster A (40 classi), Cluster B (40 classi), 5 docenti-ponte. Senza Lagrangian, A e B vanno risolti insieme: 80 classi. Con Lagrangian, vengono risolti separatamente in ~ 3 iterazioni: ogni cluster diventa 40 classi (molto più veloce), e λ converge a valori che impediscono ai docenti-ponte di sovrapporsi.

B.11 Column Generation (Dantzig-Wolfe)

Cos'è e perché esiste. Quando un problema è enorme, conviene non enumerarlo tutto: ne generi solo le parti che servono, on-demand. Column Generation è questa filosofia applicata alla programmazione lineare: hai un problema “master” che sceglie da un catalogo di colonne (soluzioni candidate); a ogni iterazione un sotto-problema genera nuove colonne “promettenti” (con *reduced cost* negativo) e le aggiunge al catalogo.

Nel timetabling: ogni docente ha un catalogo di possibili “settimane-tipo” (orari completi per quel docente, già feasible localmente). Il master sceglie quale settimana-tipo prendere per ognuno; il sotto-problema (uno per docente) genera nuove settimane-tipo guidate dal duale del master.

Come funziona. Loop: (1) il master sceglie una combinazione corrente di colonne; (2) calcola i prezzi duali sui vincoli di copertura; (3) ogni sotto-problema produce la nuova colonna più attraente data quei prezzi; (4) se ce n'è una con reduced cost negativo, aggiungila e ripeti; altrimenti hai trovato l'ottimo del rilassamento LP.

Quando usarlo. Solo per scuole molto grandi (>200 classi). Default OFF.

Esempio. 250 classi, 180 docenti. Enumerare tutte le settimane-tipo possibili sarebbe astronomicamente troppo; Column Generation parte con 3 settimane-tipo per docente (540 colonne), ne aggiunge ~50 per iterazione, e converge in 5-10 iterazioni a un orario ottimale rilassato.

B.12 Monte Carlo Sensitivity

Cos'è e perché esiste. Quando hai una soluzione, ti chiedi: *è già vicina al meglio possibile, o c'è ancora margine?* La sensitivity analysis Monte Carlo risponde generando N varianti random della soluzione (ognuna ottenuta applicando 30 mosse atomiche random feasible) e misurando il loro SOFT score. Se la varianza è piccola, sei vicino a un minimo locale; se è grande, hai ancora margine.

Come funziona. Genera N (default 100) walk random di 30 mosse ciascuno, accettando solo quelle che restano HARD-feasible. Per ognuno calcola SOFT. Riporta media, deviazione standard, percentili, coefficiente di variazione $CV = \sigma/\mu$.

Quando usarlo. Dopo aver lanciato la pipeline, per decidere se vale la pena spendere altro tempo in metaeuristiche. $CV < 0.05$ = stai già sull'ottimo locale; $CV > 0.15$ = c'è potenziale di miglioramento.

Esempio. La tua soluzione corrente ha SOFT=400. Monte Carlo restituisce media=402, std=8, $CV \approx 0.02$. Conclusione: la pipeline ha già trovato un buon minimo locale; nuovi giri non porteranno miglioramenti significativi.

B.13 Modularità, betweenness, densità (analisi del bipartito)

Cos'è e perché esiste. Tre metriche dalla teoria dei grafi che ti dicono quanto la struttura della tua scuola sia “decomponibile”. Sono interessanti perché predicono quanto certe tecniche (decomposizione spettrale, parallelismo) funzioneranno bene.

Cosa significano in pratica.

- **Modularità (Newman-Girvan):** misura quanto una partizione *già calcolata* è “naturale”. Valore in $[-1, 1]$. > 0.4 = cluster molto netti, la decomposizione spettrale è perfetta. < 0.2 = la scuola è troppo intrecciata, la decomposizione produrrà cluster artificiosi.
- **Betweenness centrality:** per ogni nodo, conta in quanti percorsi minimi del grafo passa. I nodi ad alta betweenness sono i “ponti”: se hanno SOFT alto, fanno collassare la qualità globale.

- **Densità:** $\frac{2|E|}{|V|(|V|-1)}$, frazione di archi presenti rispetto al massimo. Bassa densità (< 0.1) = scuola sparsa, decomposizione ottima. Alta densità (> 0.5) = quasi cricca, decomposizione inutile.

Quando usarli. Diagnostici da consultare prima di investire in metaeuristiche pesanti: ti dicono che strategia ha senso.

Esempio. Liceo medio: modularità 0.43, betweenness top = “Lingua straniera” (5 prof condivisi su tutti gli indirizzi), densità 0.18. Diagnosi: la decomposizione spettrale produrrà 4 cluster netti (Liceo Classico, Liceo Scientifico, Liceo Linguistico, ITIS); i prof di lingua sono i “ponti” da gestire con cura (Lagrangian relaxation aiuta).

B.14 Distribuzioni e istogrammi

Cosa misurano. 5 viste statistiche dell’orario prodotto: distribuzione del carico orario dei docenti, distribuzione del carico delle classi per giorno, heatmap materia $\times$ slot, occupazione per slot, test di goodness-of-fit (KS contro uniforme, chi-quadro per giornate).

Cosa significano in pratica.

- *Carico docenti:* se la distribuzione è schiacciata su un numero (es. tutti 18) il sistema sta rispettando i tetti di cattedra. Se ha una coda lunga (un prof con 22, un altro con 12), c’è sbilanciamento.
- *Heatmap materia $\times$ slot:* zone scure dove cadono troppe ore di una stessa materia in slot adiacenti.
- *KS-test contro uniforme:* $p > 0.05$ = i carichi sono compatibili con una distribuzione uniforme (giusto); $p < 0.05$ = c’è una distribuzione non casuale (alcuni prof troppo carichi).

Quando usarli. Per dare un giudizio qualitativo sull’orario prodotto, oltre al solo SOFT score numerico.

B.15 Correlazioni e regressioni

Cosa misurano. Tre regressioni statistiche sull’orario corrente:

1. OLS: ore di carico docente $\rightarrow$ numero di buchi.
2. OLS: numero di materie diverse del docente $\rightarrow$ contribuzione SOFT.
3. Logit: saturazione classe $\rightarrow$ probabilità di sesta ora.

Per ognuna ottieni coefficienti, p-value, R^2 .

Cosa significano in pratica. “I docenti con carico > 20 ore hanno in media 2 buchi in più ($p < 0.001$, $R^2 = 0.6$)” = c’è un effetto sistematico documentabile. “La saturazione classe non predice la 6^a ora ($p = 0.4$)” = il sistema sta distribuendo bene le ore e non concentra le classi sature in 6^a.

Quando usarle. Per riportare al collegio docenti dati *quantitativi* sulla qualità dell’orario, anziché solo affermazioni qualitative.

B.16 DSL generico

Cos’è e perché esiste. I sub-DSL precedenti (query lista, logical DNF, objective Phase A, introspettivo) risolvevano ognuno un caso, ma erano grammatiche separate. Ogni fix andava

replicato 4 volte. Il *DSL generico* unifica: una grammatica sola, parser unico, quattro back-end di compilazione.

Come si rapporta al modello dati. Le sorgenti iterabili (*lessons*, *teachers*, *classes*, *classrooms*, *students*, *groups*, ...) sono viste pre-calcolate del modello SQLAlchemy: a ogni valutazione, il sistema “materializza” queste viste in dict Python che il parser interroga.

Come si traduce. Un `forall l in lessons where ...: predicate` si traduce, per il backend EVAL, in un ciclo Python su tutti gli elementi che soddisfano il filtro, applicando il predicato. Per il backend LOGIC (vincoli logical DNF) si traduce in clausole disgiuntive di letterali. Per il backend OBJECTIVE si traduce in una somma pesata aggiunta alla funzione obiettivo della Phase A. Stessa sintassi, traduzioni diverse.

Quando usarlo. Per qualunque vincolo che non rientri nelle preferenze standard. Vedi capitolo dedicato per la gallery di 30+ esempi.

B.17 Stati 5-colori delle matrici

Cos'è. Le matrici di disponibilità (docente, classe, aula) hanno 5 stati ortogonali per ogni cella, con colori distintivi per leggerli a colpo d'occhio:

- **HARD** (rosso): cella vietata in modo assoluto.
- **ENFORCED** (rosso scuro): cella che *deve* esserci occupata.
- **PREFERRED** (verde): preferenza positiva (bonus).
- **DISLIKED/SOFT** (giallo): preferenza negativa (penalità).
- **ALLOWED** (bianco): neutro.

Scorciatoie tastiera. Selezionata una cella o un range, premere H/E/P/D/A la imposta in HARD/ENFORCED/ PREFERRED/DISLIKED/ALLOWED. N per “neutro” (alias ALLOWED). Click + drag per selezionare un range; doppio click sull'header di colonna o riga per applicare a tutta la colonna/riga.

B.18 Tag aule e tag studenti

Cos'è. Etichette libere che il coordinatore attacca a un'aula o a uno studente. Sono *many-to-many*: un'aula può avere tanti tag, un tag può essere su tante aule.

Perché esistono. I tag rendono i vincoli più robusti al cambiamento dell'anagrafica. Invece di scrivere “solo queste tre aule precise” (lista che andrebbe aggiornata a ogni modifica), scrivi “aule taggate matematica”: se domani aggiungi una nuova aula taggata allo stesso modo, il vincolo la include automaticamente.

Casi d'uso tipici.

- Tag aule: *matematica*, *proiettore*, *lab_lingue*, *piano_terra*, *accessibile_disabili*.
- Tag studenti: *BES* (Bisogni Educativi Speciali), *DSA*, *debito_matematica_4*, *pcto_Acme*, *studente_atleta*.

I tag NON sostituiscono i gruppi articolati (che sono unità operative di scheduling); sono metadato passivo per filtrare e raggruppare nell'interfaccia.

B.19 Run telemetry e visualizzazione live

Cos'è. Ogni esecuzione del solver produce una *serie temporale* di sample: ad ogni iterazione il sistema annota timestamp, fase corrente, valore obiettivo, mosse accettate/rifiutate, vincoli HARD violati. Tutto salvato in DB e accessibile dal detail page del run.

Cosa vedi. Un grafico interattivo (Apache ECharts) con l'evoluzione del SOFT score nel tempo, una linea diversa per ogni stage (LNS in azzurro, SA in verde, TS in giallo, ...), markers verticali sui cambi di stage. Statistiche aggregate per stage in tabella (n. iterazioni, durata, best obj). Esportazione in JSON per analisi esterne.

Live mode. Per i run in corso, la pagina poll-a il backend ogni 2s e aggiorna il grafico man mano che arrivano nuovi sample. La pagina pausa il polling automaticamente quando la tab del browser è in background, per non sprecare banda.

B.20 Lockati: lezioni che non si toccano

Cos'è. Il flag `locked` sulla tabella `Lesson` indica che quella lezione *non deve essere spostata* dal solver. Utile per fissare a mano una manciata di lezioni critiche e poi lanciare l'ottimizzazione su tutto il resto.

Come si usa. Dal Monitor (e dalla griglia Schedule con click-destro), il bottone “Blocca” marca la lezione come locked. Il bottone “Sblocca” fa l'inverso. Le lezioni locked sopravvivono ai run successivi: vengono caricate come *vincoli aggiuntivi HARD* dal solver, che dovrà incastrare il resto attorno a esse.

Il filtro “Lockati” nella tab del Monitor mostra solo gli eventi che hanno almeno una lezione lockata, utile per visualizzare velocemente cosa hai già fissato e cosa è ancora libero.

C. Reference

- Pisinger, "Where are the hard knapsack problems?", 2005.
- Kohl, Karisch, Patriksson, "Very Large-Scale Neighborhood Search Techniques", 2002.
- Burke, Kingston, Pepper, "A standard data format for timetabling instances", 2008 (S1877050919318800).
- Ahuja, Magnanti, Orlin, "Network Flows", 1993.